

CSL302: Compiler Design

Code Generation and Optimizations

Vishwesh Jatala

Assistant Professor

Department of CSE

Indian Institute of Technology Bhilai

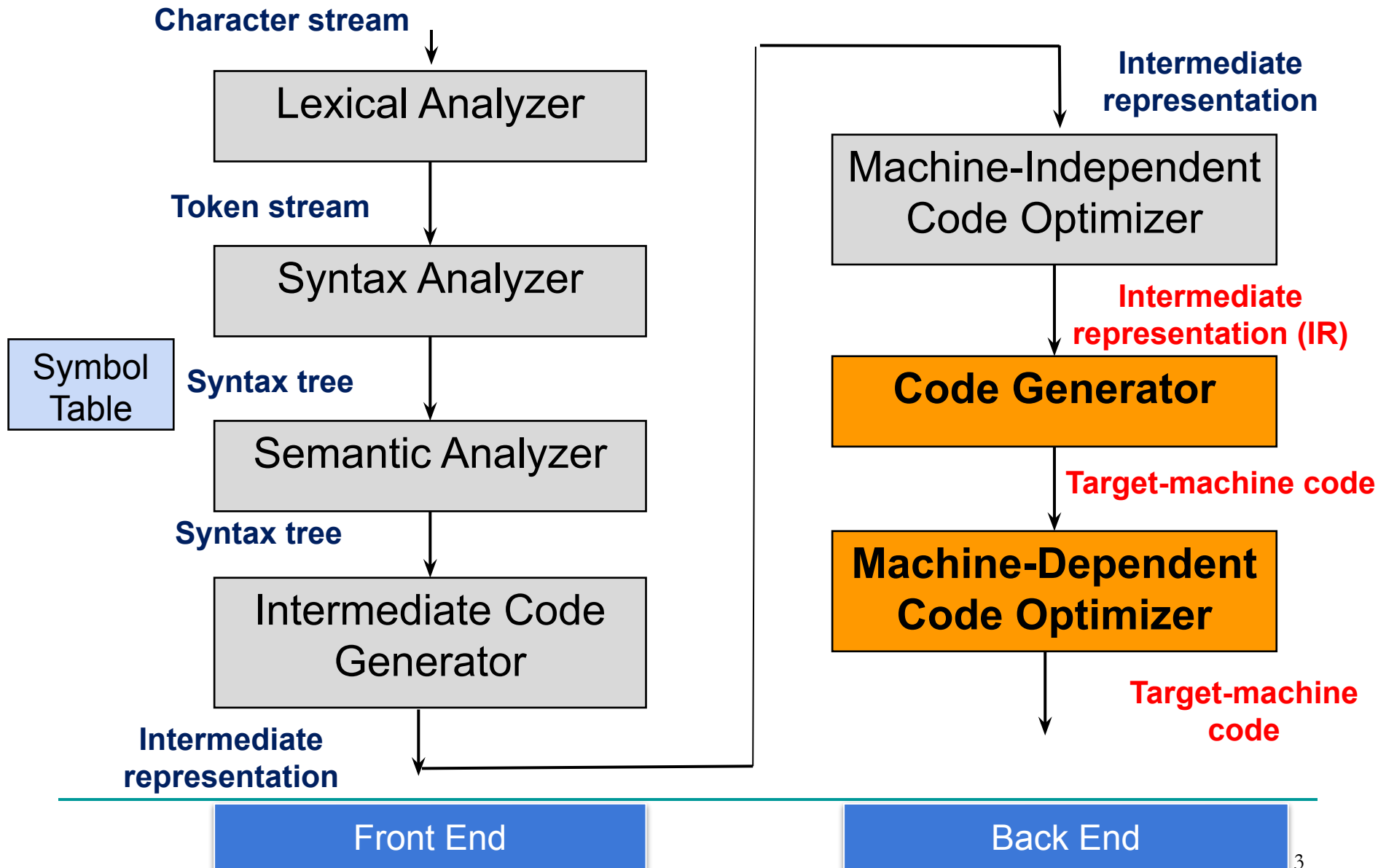
vishwesh@iitbhilai.ac.in



Acknowledgement

- References for today's slides
 - ▣ *Prof. Y. N Srikant, IISc Bangalore*
 - *<https://nptel.ac.in/content/storage2/courses/106108052/module4/code-gen-part-3.pdf>*
 - ▣ *Course textbook*

Compiler Design

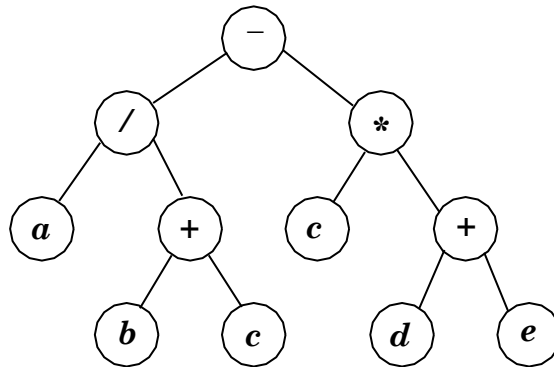


Sethi-Ullman Algorithm – Introduction

- Generates optimal code for expression trees.
 - Basic blocks containing single expression evaluation
- Target machine model is simple. Has
 - a load instruction,
 - a store instruction, and
 - binary operations involving either a register and a memory, or two registers.

Expression Trees

- Here is the expression $a/(b + c) - c * (d + e)$ represented as a tree:



Target Machine Model

- We assume a machine with finite set of registers $r_0, r_1, \dots, r_k$, countable
 1. $m \leftarrow r$ (store instruction)
 2. $r \leftarrow m$ (load instruction)
 3. $r \leftarrow r \text{ op } m$ (the result of $r \text{ op } m$ is stored in r)
 4. $r_2 \leftarrow r_2 \text{ op } r_1$ (the result of $r_2 \text{ op } r_1$ is stored in r_2)

Note:

1. In instruction 3, the memory location is the right operand.
2. In instruction 4, the destination register is the same as the left operand register.

Overview

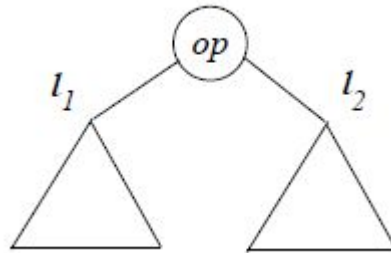
- Computes the minimum number of registers required to compute the expression tree -- labelling algorithm
- Generates the code

Overview

- Computes the minimum number of registers required to compute the expression tree -- labelling algorithm
- Generates the code

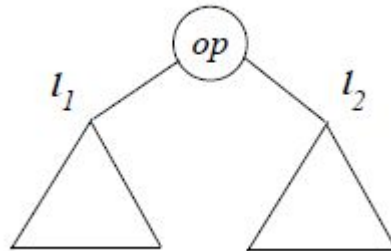
Order of Evaluation and Register Requirement

- Consider evaluation of a tree without stores. Assume that the left and right subtrees require up to l_1 , and l_2 ($l_1 < l_2$) registers.
- In what order should we evaluate the subtrees to minimize register requirement?



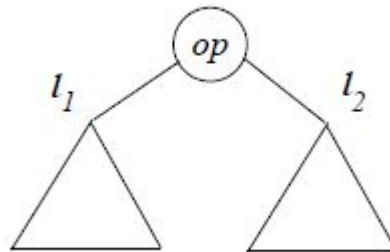
Order of Evaluation: Choice-1

- Left subtree first, leaving result in a register. This requires upto l_1 registers.
- Evaluate the right subtree. During this we require upto l_2 for evaluating the right subtree and one to hold value of the left subtree.
- Register requirement — $\max(l_1, l_2 + 1) = l_2 + 1$.



Order of Evaluation: Choice-2

- Evaluate the right subtree first, leaving the result in a register. During this evaluation we shall require up to l_2 registers.
- Evaluate the left subtree. During this, we might require up to $l_1 + 1$ registers.
- Register requirement — $\max(l_1 + 1, l_2) = l_2$



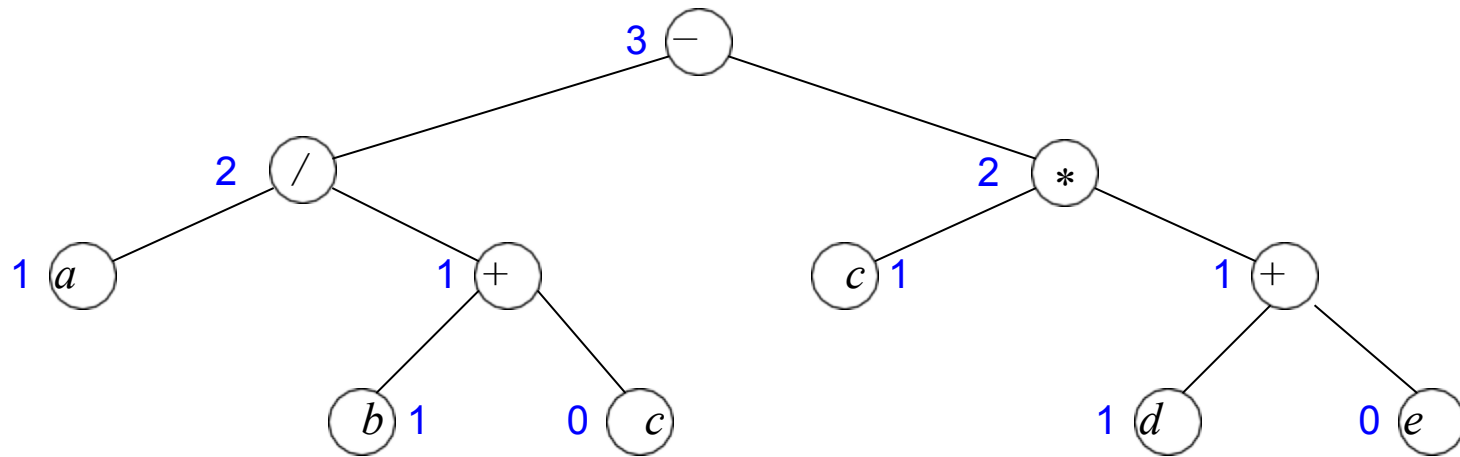
Therefore the subtree requiring more registers should be evaluated first.

Labeling the Expression Tree

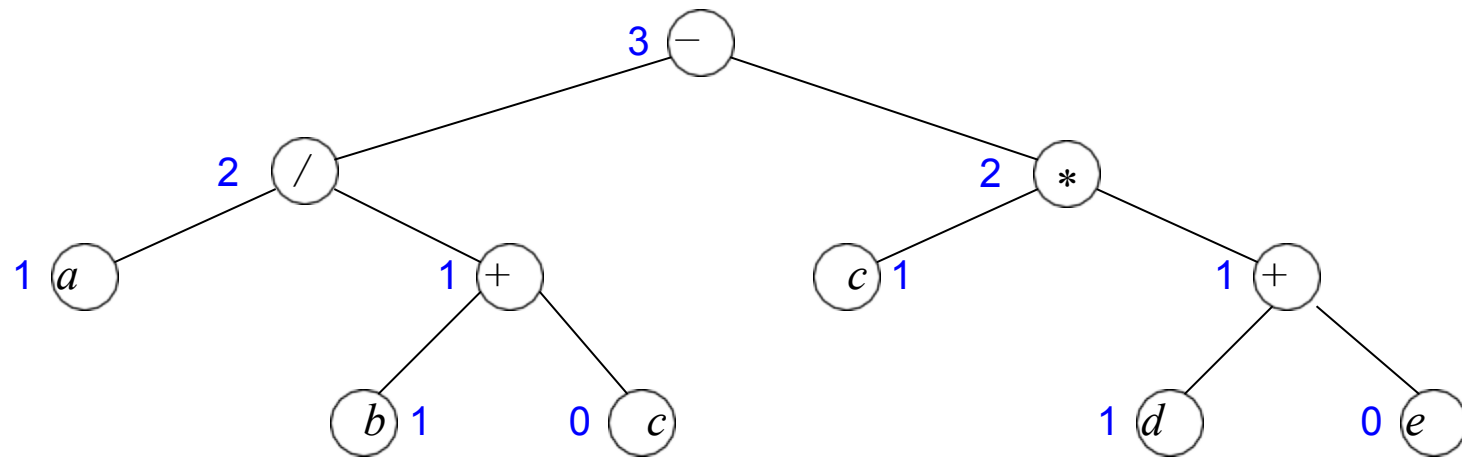
- Label each node by the number of registers required to evaluate it in a store free manner.
- Visit the tree in post-order. For every node visited do:
 1. Label each left leaf by 1 and each right leaf by 0.
left leaf must necessarily be in a register
the right leaf can reside in memory.
 1. If the labels of the children of a node n are l_1 and l_2 respectively, then
$$\begin{aligned} \text{label}(n) &= \max(l_1, l_2), \text{ if } l_1 \neq l_2 \\ &= l_1 + 1, \text{ otherwise} \end{aligned}$$

Labeling the Expression Tree

$$a/(b + c) - c * (d + e)$$



Labeling the Expression Tree



Exercise

- Computes the minimum number of registers required to compute the expression tree using labelling algorithm

$$((a+b)*(c-d*e))+((f*g)/(h+i))$$

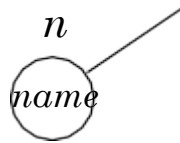
Assumptions and Notational Conventions

1. The code generation algorithm is represented as a function $gencode(n)$, which produces code to evaluate the node labeled n .
2. Register allocation is done from a stack of register names $rstack$, initially containing $r_0, r_1, \dots, r_k$ (with r_0 on top of the stack).
3. $gencode(n)$ evaluates n in the register on the top of the stack.
4. Temporary allocation is done from a stack of temporary names $tstack$, initially containing $t_0, t_1, \dots, t_k$ (with t_0 on top of the stack).
5. $swap(rstack)$ swaps the top two registers on the stack.

The Algorithm

$gencode(n)$ described by case analysis on the type of the node n .

1. n is a left leaf:

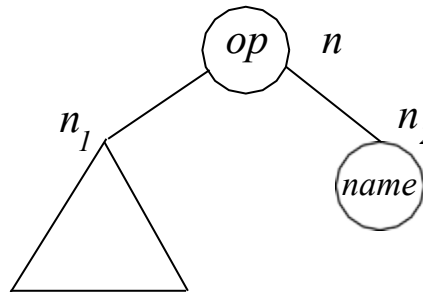


$gen(top(rstack)) \leftarrow name$

Comments: n is named by a variable say *name*. Code is generated to load *name* into a register.

The Algorithm

2. n 's right child is a leaf:

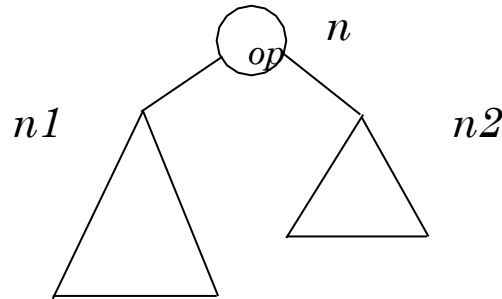


$gencode(n_1);$
 $gen(top(rstack)) \leftarrow top(rstack) \text{ op } name$

Comments: n_1 is first evaluated in the register on the top of the stack, followed by the operation op leaving the result in the same register.

The Algorithm

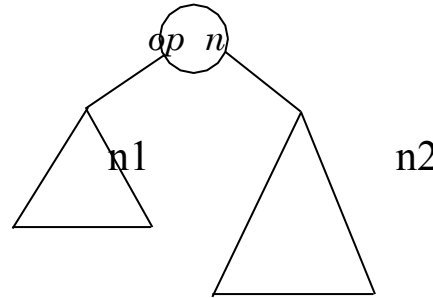
3. *The right child of n is lighter or as heavy as the left child. Its requirement is strictly less than the available number of registers*



```
gencode( $n_1$ );  
 $R := pop(rstack)$ ;  
gencode( $n_2$ );  
gen( $R \leftarrow R \text{ op } top(rstack)$ );  
push( $rstack$ ,  $R$ )
```

The Algorithm

4. *The left child is the lighter subtree. This requirement is strictly less than the available number of registers*



swap(rstack);

gencode(n₂);

R := pop(rstack);

gencode(n₁);

gen(top(rstack) ← top(rstack) op R);

push(rstack, R);

swap(rstack)

Evaluate right child

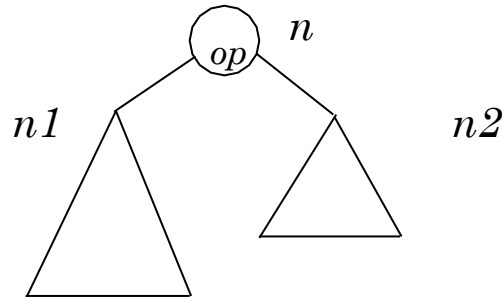
Evaluate left child

Issue *op*

Restore register stack

The Algorithm

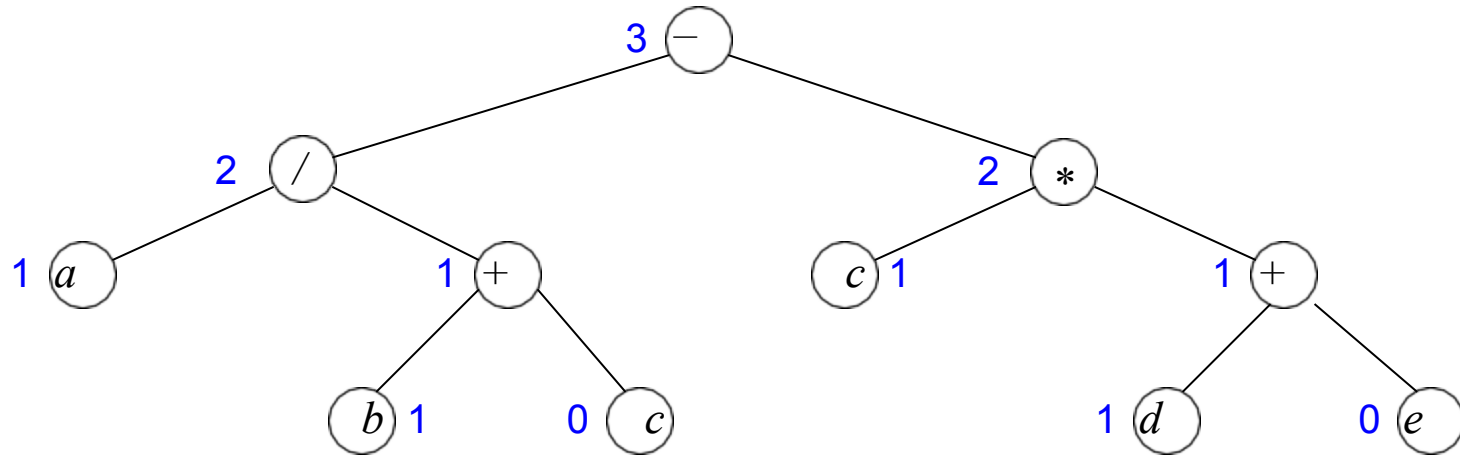
5. Both the children of n require registers greater or equal to the available number of registers.



```
gencode( $n_2$ );  
 $T := \text{pop}(\text{tstack});$   
 $\text{gen}(T \leftarrow \text{top}(\text{rstack}));$   
 $\text{gencode}(n_1);$   
 $\text{push}(\text{tstack}, T);$   
 $\text{gen}(\text{top}(\text{rstack}) \leftarrow \text{top}(\text{rstack}) \text{ op } T);$ 
```

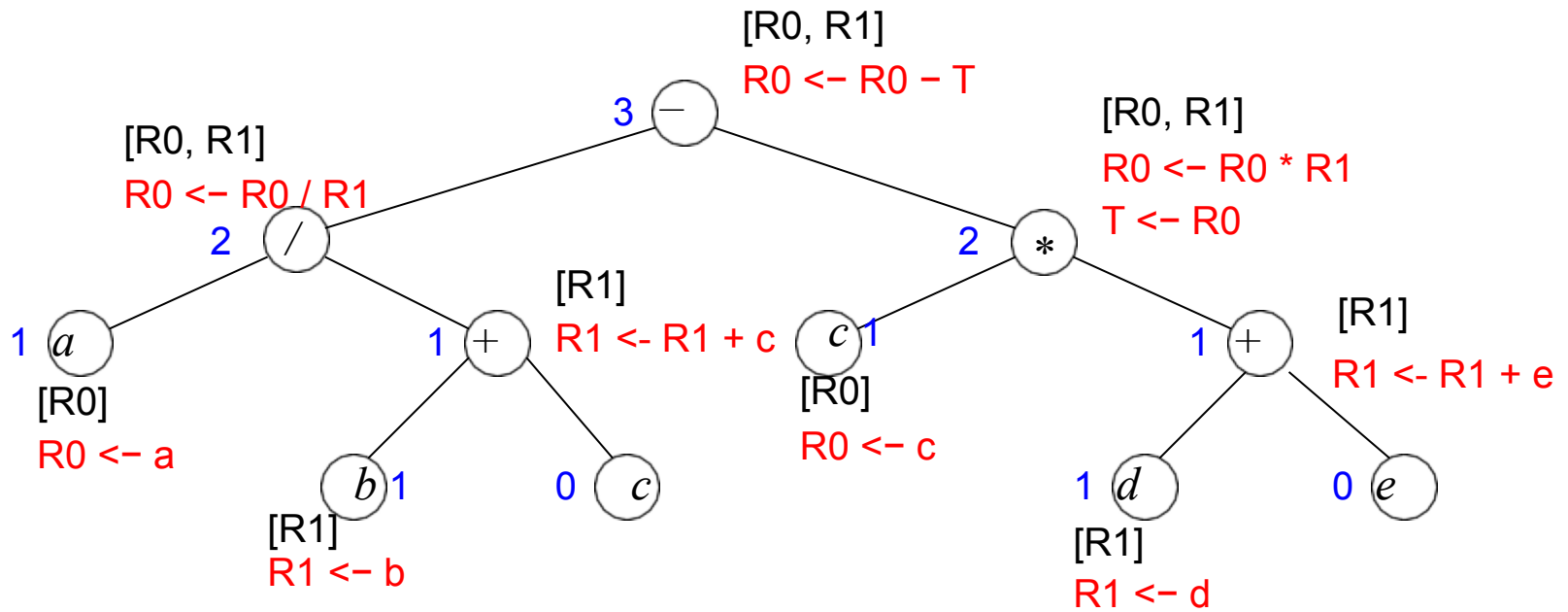
Comments: Evaluate the right sub-tree into a temporary. Then evaluate the left sub-tree and n into the register on top of stack.

Example

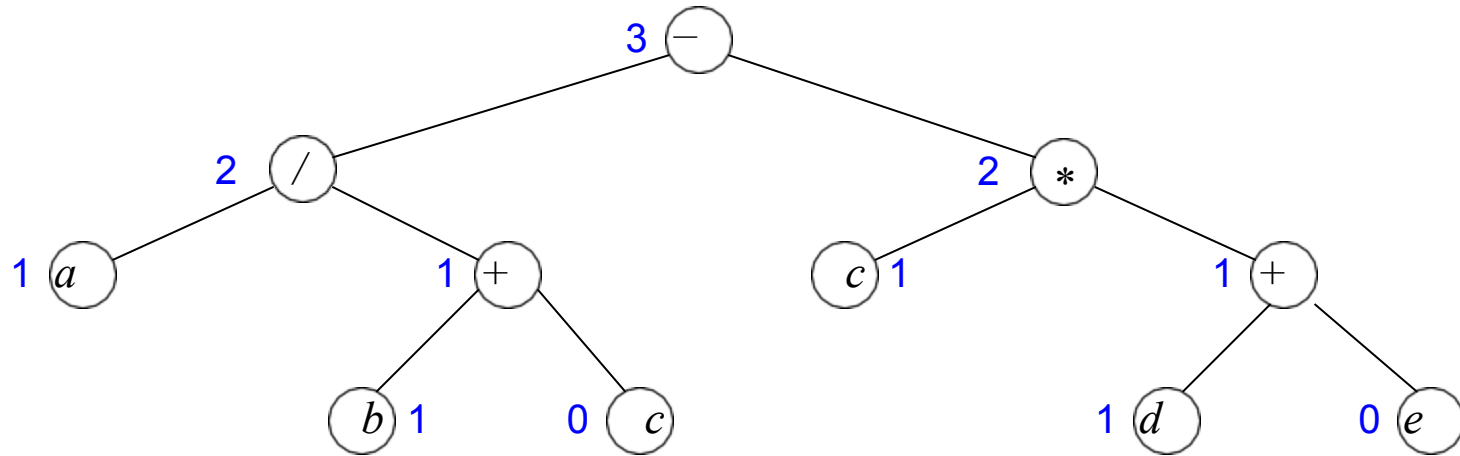


assuming two available registers r_0 and r_1 , the calls to gencode and the generated code are shown below.

Example



Exercise-1



Assuming *three* available registers r_0 , r_1 and r_2 what would be the generated code?

Peephole Optimizations

Peephole Optimizations

- Simple but effective local optimization
- Usually carried out on machine code, but intermediate code can also benefit from it
- Examines a sliding window of code (peephole), and replaces it by a shorter or faster sequence, if possible
- Each improvement provides opportunities for additional improvements
- Therefore, repeated passes over code are needed

Peephole Optimizations

- Some well known peephole optimizations
 - ❑ eliminating redundant instructions
 - ❑ eliminating unreachable code
 - ❑ eliminating jumps over jumps
 - ❑ algebraic simplifications
 - ❑ strength reduction
 - ❑ use of machine idioms

Elimination of Redundant Loads and Stores

Basic block B

Load X, R0
{no modifications
to R0 or X here}
Store R0, X

Store instruction
can be deleted

Basic block B

Load X, R0
{no modifications
to X or R0 here}
Load X, R0

Second Load instr
can be deleted

Basic block B

Store R0, X
{no modifications
to X or R0 here}
Load X, R0

Load instruction
can be deleted

Basic block B

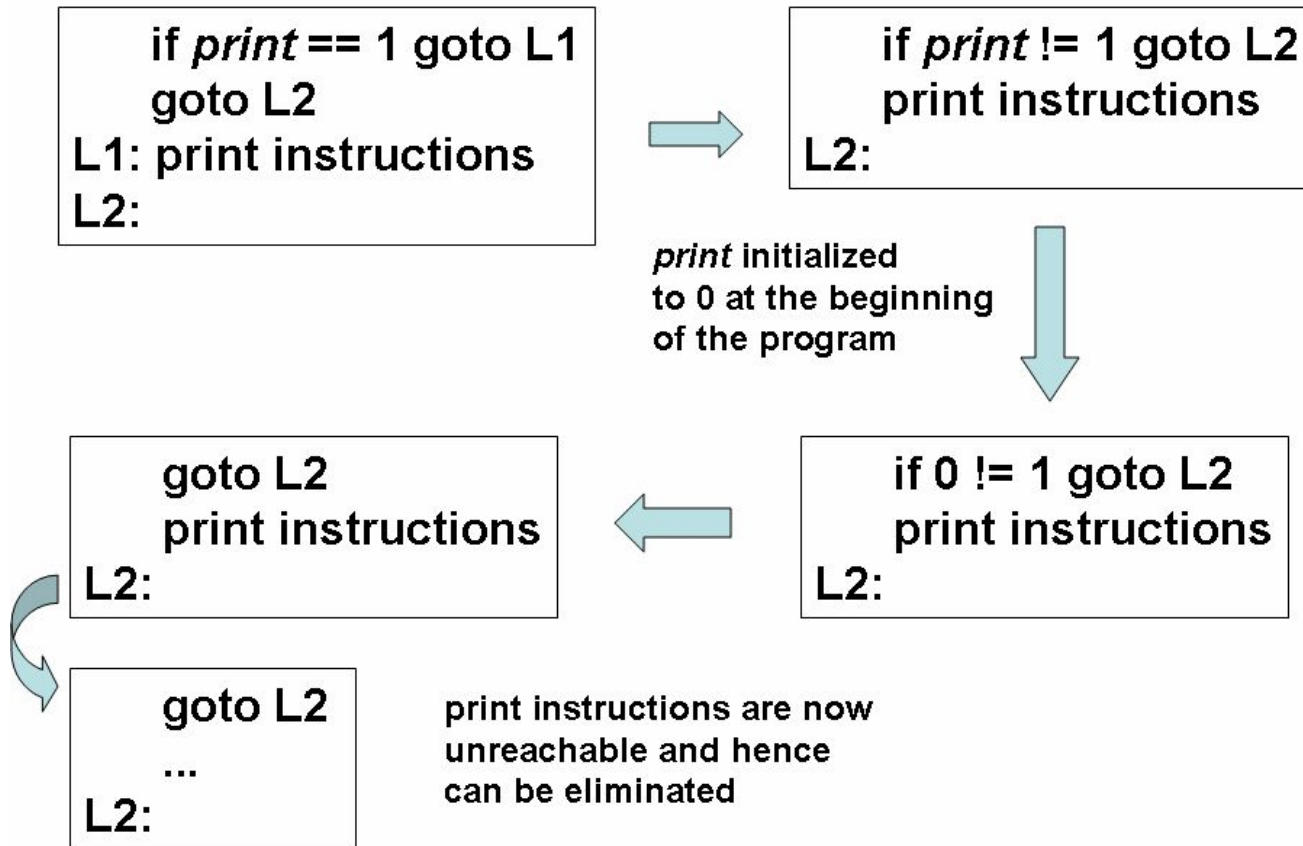
Store R0, X
{no modifications
to X or R0 here}
Store R0, X

Second Store instr
can be deleted

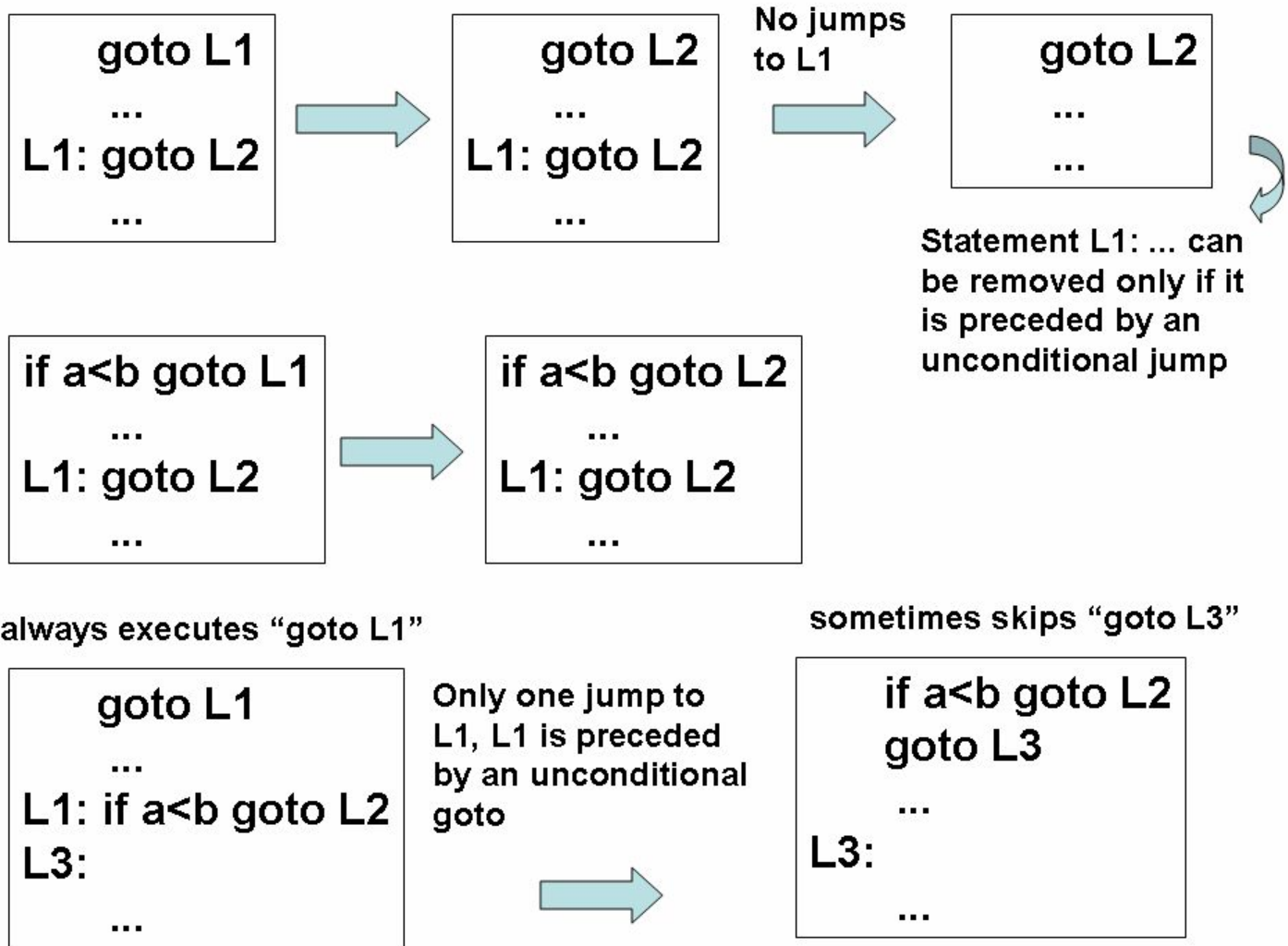
Eliminating Unreachable Code

- An unlabeled instruction immediately following an unconditional jump may be removed
 - May be produced due to debugging code introduced during development

Eliminating Unreachable Code



Flow-of-Control Optimizations



Reduction in Strength and Use of Machine Idioms

- x^2 is cheaper to implement as $x*x$, than as a call to an exponentiation routine
- For integers, $x*2^3$ is cheaper to implement as $x \ll 3$ (x left-shifted by 3 bits)
- For integers, $x/2^2$ is cheaper to implement as $x \gg 2$ (x right-shifted by 2 bits)

Summary

- Code generation algorithm
 - ▣ Sethi-Ullman Algorithm
- Peephole optimizations

Summary of Course

